

1. Lec 1-3, slide 35 (Generating coordinates of a logarithmic spiral)
2. Lec 1-3, slide 36 (Generating coordinates of a lemniscate)
3. Lec 1-3, slide 38 (Construction of large Matrix and element extraction)
4. Lec 1-3, slide 52 (Use of Recursive loop to find the nth Fibonacci number)
5. Lec 4-6, slide 8 (Bisection Method For $\sqrt{3}$)
6. Lec 4-6, slide 13 (Newtons Method For $\sqrt{2}$)
7. Lec 4-6, slide 22 (Secant Method For $\sqrt{2}$)
8. Lec 7, slide 7 (Newton's Fractal for Cube roots of Unity)
9. Lec 8-11, slide 6 (Using Kepler's Equation to plot the Satellite elliptical transfer orbit)
10. Lec 12-15, slide 56 (Eigenvalues and Eigenvectors)
11. Lec 19-20, slide 32 (Finding Integral value)
12. Lec 19-20, slide 33 (Fresnel Integral)
13. Lec 21-27, slide 10 (Interpolation Comparison)
14. Lec 28-30, slide 10 (Lorenz Equation)
15. Lec 30-33, slide 18,27,28 (Laplace Equation)

Details Given Below.

PB-01 Lec 1-3, slide 35 (Generating coordinates of a logarithmic spiral)

Generate the coordinates of a lemniscate, with

$$X = e^{k\theta} \cos(\theta),$$

$$Y = e^{k\theta} \sin(\theta)$$

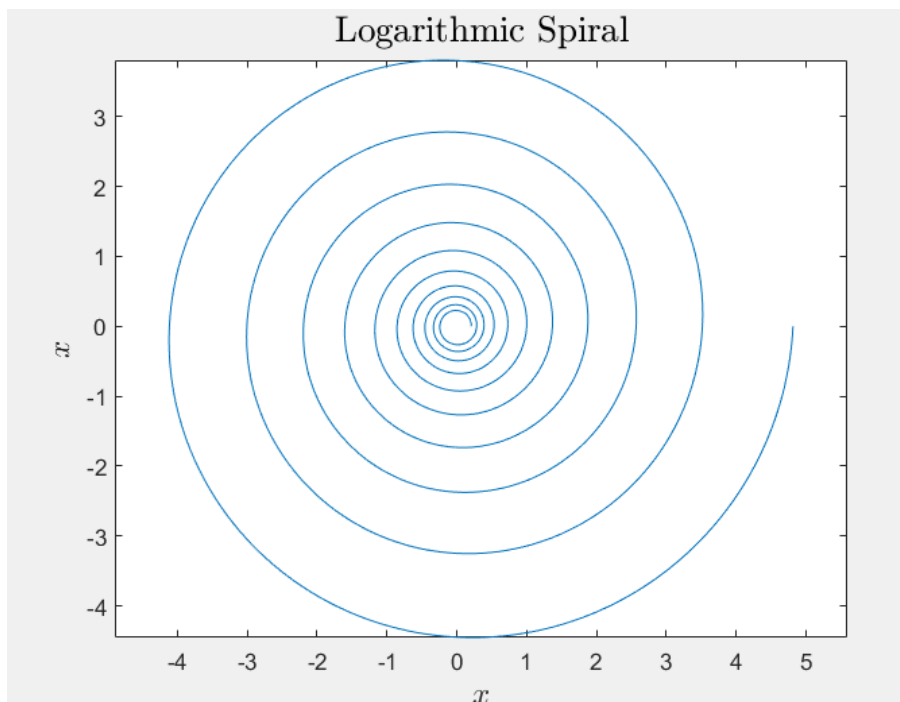
Choose $k=0.05$ and plot θ values from -10π to 10π using 2000 evenly spaced point

Code-

```
k=0.05;
theta= linspace(-10*pi,10*pi,2000);
x= exp(k*theta).*cos(theta);
y= exp(k*theta).*sin(theta);

plot(x,y);
axis equal;
xlabel('$x$', 'Interpreter', 'latex', 'FontSize', 14);
ylabel('$y$', 'Interpreter', 'latex', 'FontSize', 14);
title('Logarithmic Spiral', 'Interpreter', 'latex', 'FontSize', 16);
```

output-



PB-02 Lec 1-3, slide 36(Generating coordinates of a lemniscate)

Generate the coordinates of a lemniscate, with

$$X = \pm \cos\theta \sqrt{2\cos(2\theta)}, \quad y = \sin\theta \sqrt{2\cos(2\theta)}$$

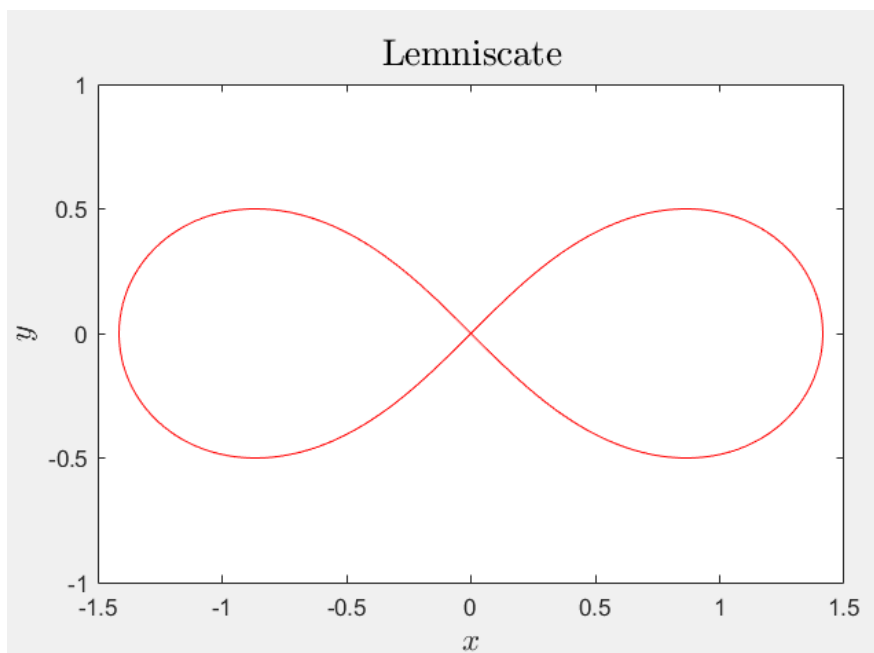
Plot θ values from $-\pi/4$ to $\pi/4$ using 1000 evenly spaced points.

CODE:

```
% Assign theta, x, and y
theta = linspace(-pi/4, pi/4, 1000); % 1000 evenly spaced points from
-π/4 to π/4
x = cos(theta) .* sqrt(2 .* cos(2*theta)); % x-coordinates (positive
branch)
y = sin(theta) .* sqrt(2 .* cos(2*theta)); % y-coordinates

% Graphics
plot(x, y, 'r', -x, y, 'r') % Plot both positive and negative branches
of x
axis equal;
axis([-1.5 1.5 -1 1])
xlabel('$x$', 'Interpreter', 'latex', 'FontSize', 14);
ylabel('$y$', 'Interpreter', 'latex', 'FontSize', 14);
title('Lemniscate', 'Interpreter', 'latex', 'FontSize', 16);
```

OUTPUT:



PB-03 Lec 1-3, slide 38 (Construction of large Matrix and element extraction)

Construct the following matrix in MATLAB. Use the colon operator and the ones.m function:

$$A = \begin{bmatrix} 6 & 9 & 12 & 15 & 18 & 21 \\ 4 & 4 & 4 & 4 & 4 & 4 \\ 2 & 1 & 0 & -1 & -2 & -3 \\ -6 & -4 & -2 & 0 & 2 & 4 \end{bmatrix}$$

Evaluate the following expressions without using MATLAB. Check your answers with MATLAB.

a) $A([1, 3], [2, 4])$

b) $A(:, [1, 4:6])$

c) $A([2, 3], :)$

CODE:

```
% Construct the matrix A using the colon operator and ones.m
A = [6:3:21; 4 * ones(1, 6); 2:-1:-3; -6:2:4];
% Perform the operations
B = A([1,3],[2,4]); % Extract elements from rows 1 and 3,
columns 2 and 4
C = A(:,[1,4:6]); % Extract all rows, columns 1 and 4 to 6
D = A([2,3],:); % Extract rows 2 and 3,
all columns
% Display the results
disp('Matrix A:');
disp(A);
disp('B = A([1,3],[2,4]):');
disp(B);
disp('C = A(:,[1,4:6]):');
disp(C);
disp('D = A([2,3],:):');
disp(D);
```

OUTPUT:

Matrix A:

6	9	12	15	18	21
4	4	4	4	4	4
2	1	0	-1	-2	-3
-6	-4	-2	0	2	4

B = A([1,3],[2,4]):

9	15
1	-1

C = A(:, [1,4:6]):

6	15	18	21
4	4	4	4
2	-1	-2	-3
-6	0	2	4

D = A([2,3],:):

4	4	4	4	4	4
2	1	0	-1	-2	-3

fx >>

PB-04 Lec 1-3, slide 52 (Use of Recursive loop to find the nth Fibonacci number)

Write a function that returns the n th Fibonacci number F_n for input integer n . Use the recursion relation

$$F_n = F_{n-1} + F_{n-2}; \quad F_1 = 1, F_2 = 2$$

Handle zero and negative integers using

$$F_0 = 0, \quad F_{-n} = (-1)^{n+1} F_n$$

CODE:

```
function F = Fibonacci(n)
    %returns the nth Fibonacci number using recursion relation
    n = round(n);

    if n == 0
        F = 0; %Base case: F0=0
    elseif n == 1
        F = 1; %Base case: F1 = F2 =1
    elseif n > 0
        F = Fibonacci(n-1)+ Fibonacci(n-2);
        %recursive case for positive number
    else
        %Use the property F(-n) = (-1)^(n+1)*F(n)
        F = (-1)^(n+1)*Fibonacci(abs(n));
    end
end
```

OUTPUT:

```
>> Fibonacci(9)
ans =
    34

>> Fibonacci(-30)
ans =
   -832040

>> Fibonacci(11)
ans =
    89
```

PB-05 Lec 4-6, slide 8 (Bisection Method For $\sqrt{3}$)

Using the bisection method, estimate $\sqrt{3} = 1.73205$ Use $x_0 = 1$ and $x_1 = 2$ and iterate to the value of x_4

CODE:

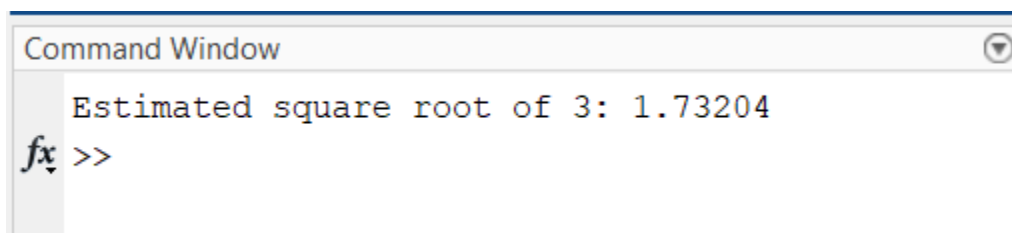
```
clc; clear; close all;
% Define function f(x) = x^2 - 3
f = @(x) x.^2 - 3;
% Set initial interval [x0, x1]
x0 = 1;
x1 = 2;
% Set tolerance (how close we want to get to the root)
tol = 1e-5;
% Maximum iterations to prevent infinite loop
max_iter = 100;

% Start bisection method
iter = 0;
while (x1 - x0) / 2 > tol && iter < max_iter
    x2 = (x0 + x1) / 2; % Midpoint
    if f(x2) == 0 % If exactly the root, stop
        break;
    elseif f(x2) * f(x0) < 0 % Root is between x0 and x2
        x1 = x2;
    else % Root is between x2 and x1
        x0 = x2;
    end

    iter = iter + 1;
end

% Display result
fprintf('Estimated square root of 3: %.5f\n',x2);
```

OUTPUT:

A screenshot of the MATLAB Command Window. The title bar says "Command Window". The main area displays the text "Estimated square root of 3: 1.73204" on a single line. Below this, the prompt "fx >>" is visible on the next line. The window has a standard Mac OS-style title bar with a close button on the right.

PB-06 Lec 4-6, slide 13 (Newtons Method For $\sqrt{2}$)

Using the newtons method, *Estimate* $\sqrt{2}=1.41421...$ using $x_0=1$ and iterate to the value of x_3

CODE:

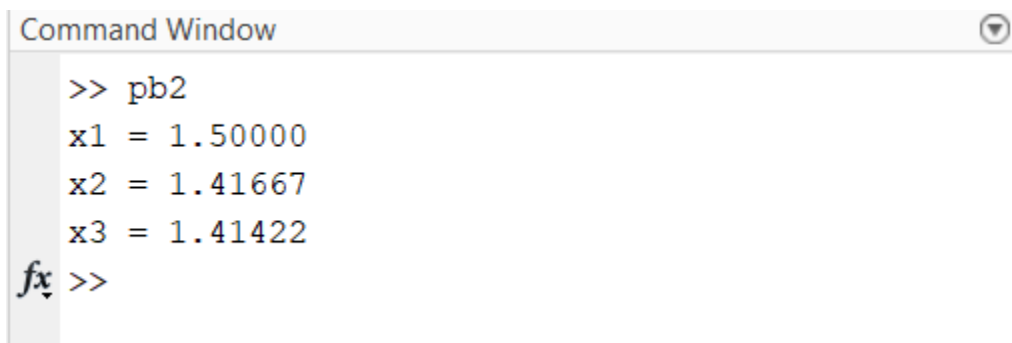
```
% Initial guess
x0 = 1;

% Number of iterations
iterations = 3;

% Define the function and its derivative
f = @(x) x^2 - 2; % f(x) = x^2 - 2
f_prime = @(x) 2*x; % f'(x) = 2*x
% Initialize the current guess
x = x0;

% Iterate using Newton's method
for n = 1:iterations
    x = x - f(x) / f_prime(x); % Newton's method formula
    fprintf('x%d = %.5f\n', n, x);
end
```

OUTPUT:



Command Window

```
>> pb2
x1 = 1.50000
x2 = 1.41667
x3 = 1.41422
fx >>
```


PB-07 Lec 4-6, slide 22 (Secant Method For $\sqrt{2}$)

Write a MATLAB code to implement the Secant method to estimate $\sqrt{2}$. Use the initial guesses $x_0 = 1$ and $x_1 = 2$ and iterate the method to x_4 .

(Lec 4-6, slide 22)

Matlab Code :

```
% Secant Method to estimate sqrt(2)

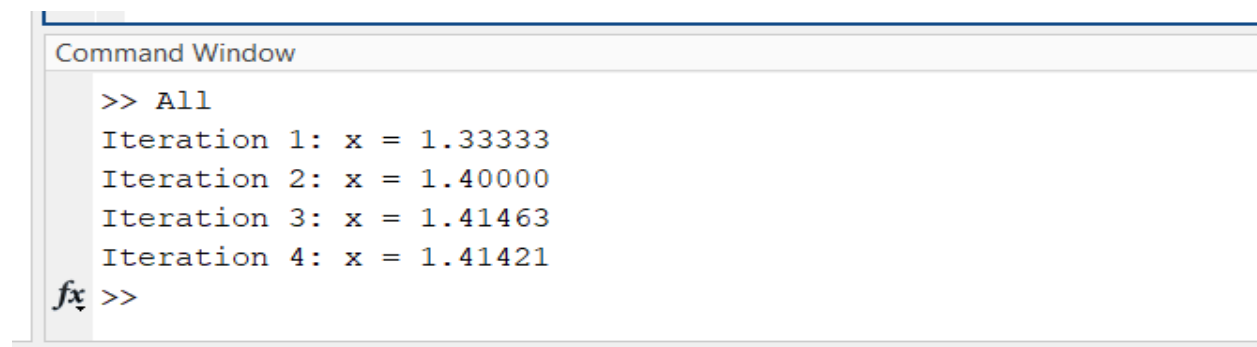
% Define the function f(x) = x^2 - 2
f = @(x) x^2 - 2;

% Initial guesses
x0 = 1;
x1 = 2;

% Number of iterations
iterations = 4;

% Secant Method loop
for n = 1:iterations
    x2 = x1 - ( (f(x1) * (x1 - x0)) / (f(x1) - f(x0)) );
    fprintf('Iteration %d: x = %.5f\n', n, x2);
    x0 = x1;
    x1 = x2;
end
```

Output:

A screenshot of the MATLAB Command Window. The title bar says "Command Window". The text inside shows the execution of the code: ">> All" followed by four lines of output: "Iteration 1: x = 1.33333", "Iteration 2: x = 1.40000", "Iteration 3: x = 1.41463", and "Iteration 4: x = 1.41421". At the bottom, there is a cursor icon and the prompt ">>".

```
Command Window

>> All
Iteration 1: x = 1.33333
Iteration 2: x = 1.40000
Iteration 3: x = 1.41463
Iteration 4: x = 1.41421
fx >>
```

PB-08 Lec 7, slide 7 (Newton's Fractal for Cube roots of Unity)

Write a MATLAB code to generate the Newton's Fractal for the cube roots of unity. The function to be used is:

$$f(z) = z^3 - 1$$

(Lec 7, slide 7)

Matlab Code:

```
clc; clear; close all;

% Define the function f(z) = z^3 - 1 and its derivative f'(z)
f = @(z) z.^3 - 1;
fp = @(z) 3*z.^2;

% Define the three cube roots of unity
root1 = 1;
root2 = -1/2 + 1i*sqrt(3)/2;
root3 = -1/2 - 1i*sqrt(3)/2;

% Define grid size and range
nx = 2000; ny = 2000;
xmin = -2; xmax = 2;
ymin = -2; ymax = 2;

% Create a 2D grid in the complex plane
x = linspace(xmin, xmax, nx);
y = linspace(ymin, ymax, ny);
[X, Y] = meshgrid(x, y);
Z = X + 1i * Y;

% Number of iterations for Newton's method
nit = 40;
for n = 1:nit
    Z = Z - f(Z) ./ fp(Z);
end

% Convergence test with tolerance
eps = 0.001;
Z1 = abs(Z - root1) < eps;
Z2 = abs(Z - root2) < eps;
```

```

Z3 = abs(Z - root3) < eps;
Z4 = ~(Z1 | Z2 | Z3); % Unclassified points (should be none)

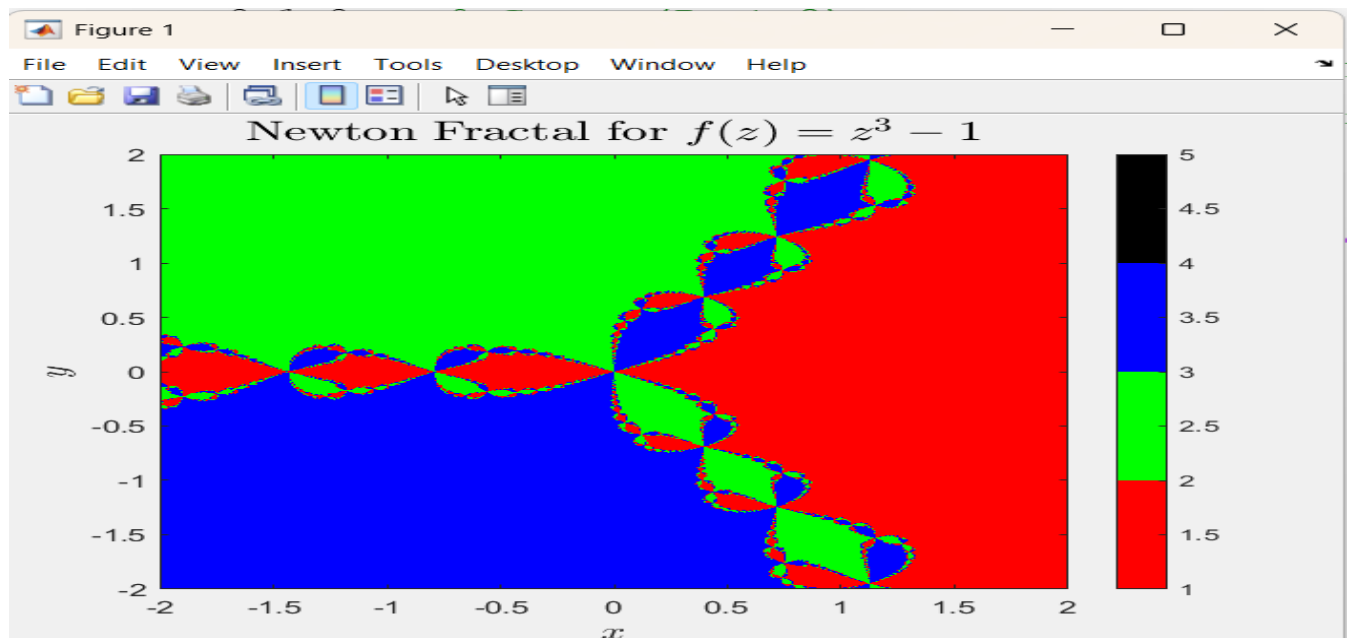
% Assign colors to each root
Z_final = ones(size(Z)) * 4; % Default to black, if needed
Z_final(Z1) = 1; % Red
Z_final(Z2) = 2; % Green
Z_final(Z3) = 3; % Blue

% Plot the fractal using the correct color mapping
figure;
colormap([1 0 0; % Red (Root 1)
          0 1 0; % Green (Root 2)
          0 0 1; % Blue (Root 3)
          0 0 0]); % Black (for undefined points, if any)
image([xmin xmax], [ymin ymax], Z_final);
set(gca, 'YDir', 'normal'); % Ensure correct orientation

% Labels and title
xlabel('$x$', 'Interpreter', 'latex', 'FontSize', 14);
ylabel('$y$', 'Interpreter', 'latex', 'FontSize', 14);
title('Newton Fractal for $f(z) = z^3 - 1$', 'Interpreter',
      'latex', 'FontSize', 16);
colorbar; % Optional: Displays color mapping for verification

```

Output:



PB-09 Lec 8-11, slide 6 (Using Kepler's Equation to plot the Satellite elliptical transfer orbit)

An Earth observation satellite is launched into an elliptical transfer orbit from a low Earth parking orbit toward a higher geostationary orbit. The motion of the satellite occurs in a fixed orbital plane with the Earth's center positioned at the origin of a two-dimensional Cartesian coordinate system. The satellite's position as a function of time is described by the following parametric equations derived from the geometry of elliptical orbits:

$$x(t) = a(\cos E - e) , y(t) = a\sqrt{1 - e^2} \sin E$$

where: a is the semi-major axis of the orbit, e is the eccentricity of the orbit ($0 < e < 1$), E is the eccentric anomaly, a time-dependent parameter satisfying Kepler's Equation: $E - e \sin E = M = \omega t$ Here, $\omega = 2\pi / T$ is the mean angular velocity and T is the orbital period.

Assumptions:

The orbit has an eccentricity of $e = 0.6$, The semi major axis is $a = 26,560$ km, The orbital period is $T = 43,200$ seconds (12 hours), All calculations are performed using SI units, The simulation covers the full time interval from $t = 0$ to $t = T$.

(Lec 8-11, slide 6)

Matlab Code:

```
% Parameters
e = 0.6; % Eccentricity
a = 26560e3; % Semi-major axis in meters (converted from km)
T = 43200; % Orbital period in seconds (12 hours)
omega = 2*pi/T; % Mean angular velocity

% Time setup
t = linspace(0, T, 1000); % Time vector
x = zeros(size(t)); % Preallocate x positions
y = zeros(size(t)); % Preallocate y positions

% Loop to calculate positions
for i = 1:length(t)
```

```

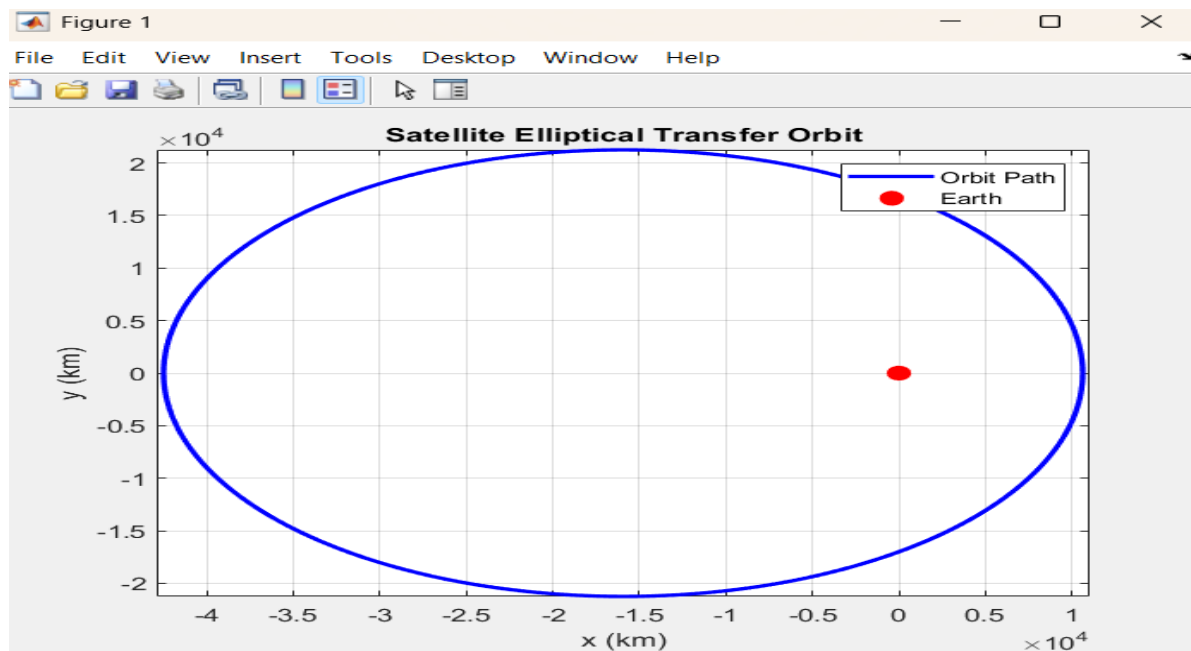
M = omega * t(i); % Mean anomaly
% Define function to solve for E
kepler_eq = @(E) E - e*sin(E) - M;
E = fzero(kepler_eq, M); % Solve Kepler's equation using
fzero

% Calculate x and y from E
x(i) = a * (cos(E) - e);
y(i) = a * sqrt(1 - e^2) * sin(E);
end

% Plotting the orbit
figure;
plot(x/1000, y/1000, 'b', 'LineWidth', 2); % convert to km
for display
hold on;
plot(0, 0, 'ro', 'MarkerSize', 8, 'MarkerFaceColor', 'r'); %
Earth at origin
axis equal;
grid on;
xlabel('x (km)');
ylabel('y (km)');
title('Satellite Elliptical Transfer Orbit');
legend('Orbit Path', 'Earth');

```

Output:



PB-10 Lec 12-15, slide 56 (Eigenvalues and Eigenvectors)

Use MATLAB to find the eigenvalues and eigenvectors of the matrix:

$$A = \begin{pmatrix} -5 & 6 \\ 5 & -4 \end{pmatrix}$$

Normalize the eigenvectors so that their second component is one.

Matlab Code:

```
% Define matrix A
A = [-5 6; 5 -4];

% Find eigenvalues and eigenvectors
[V, D] = eig(A);

% Eigenvalues (make sure lambda1 < lambda2)
lambda = diag(D);
lambda = sort(lambda); % ensure sorted

lambda1 = lambda(1);
lambda2 = lambda(2);

% Find eigenvectors associated with lambda1 and lambda2
% (Columns of V are the eigenvectors corresponding to unsorted
eigenvalues, so we re-match)
[~, idx] = sort(diag(D));
V = V(:, idx);

% Normalize eigenvectors so that their second component is
one
v1 = V(:,1) / V(2,1);
v2 = V(:,2) / V(2,2);

% Display the results
lambda1
lambda2
v1
v2
```

Output:

```
+ | Fibonacci.m | P 3.m | P 14.m | P 4.2.m | P 15.m | P 16.m |
Command Window

>> All
>> All

lambda1 =

    -10

lambda2 =

     1

v1 =

    -1.2000
     1.0000

v2 =

     1
     1

fx >> |
```

PB-11 Lec 19-20, slide 32 (Finding Integral value)

Write a MATLAB code to compute the integral value :

$$I = \int_0^{\infty} e^{-ax^2} (\ln x)^2 dx \quad \text{where } a = \pi$$

(Lec 19-20, slide 32)

Matlab Code:

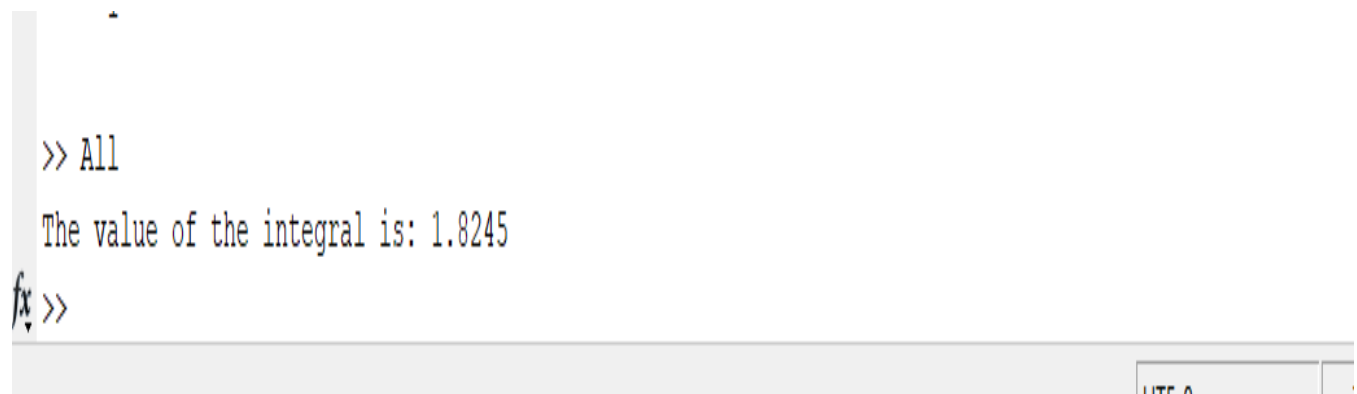
```
% Define the function f(x, a) = exp(-a*x^2) * (log(x))^2
f = @(x, a) exp(-a*x.^2) .* log(x).^2;

% Set the parameter a = pi
a = pi;

% Compute the integral using adaptive quadrature
I = integral(@(x) f(x, a), 0, Inf);

% Display the result
disp(['The value of the integral is: ', num2str(I)]);
```

Output:

A screenshot of the MATLAB command window. The window has a title bar at the top. The command prompt shows the execution of the code. The output is displayed as 'The value of the integral is: 1.8245'. The cursor is at the end of the command line, ready for the next input.

```
>> All
The value of the integral is: 1.8245
fx >>
```


PB-12 Lec 19-20, slide 33 (Fresnel Integral)

Consider the Fresnel integrals, defined by

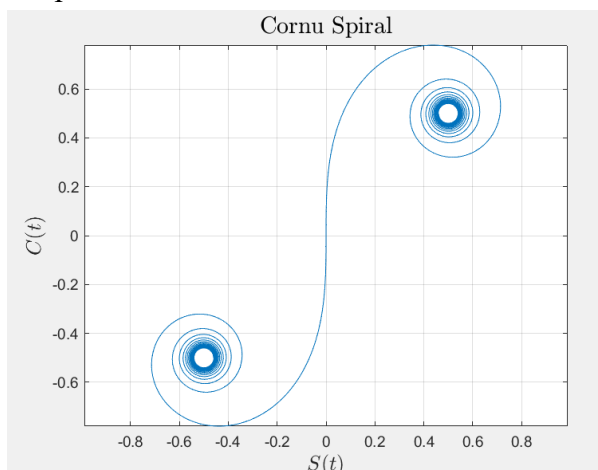
$$C(t) = \int_0^t \cos\left(\frac{1}{2}\pi x^2\right) dx, \quad S(t) = \int_0^t \sin\left(\frac{1}{2}\pi x^2\right) dx.$$

Write a script using `integral.m` to plot a Cornu spiral, which is a smooth curve of $C(t)$ versus $S(t)$. Plot your solution over the range $-8 \leq t \leq 8$.

Ans:

```
% Define the integrands for Fresnel integrals
c = @(x) cos(0.5 * pi * x.^2); % Integrand for C(t)
s = @(x) sin(0.5 * pi * x.^2); % Integrand for S(t)
% Set the range and number of sample points
tmin = -8; tmax = 8; nt = 2000;
t = linspace(tmin, tmax, nt);
% Preallocate output arrays
C = zeros(nt,1);
S = zeros(nt,1);
% Compute Fresnel integrals for each value of t
for i = 1:nt
    C(i) = integral(c, 0, t(i)); % C(t) = ??? cos(1/2?x2) dx
    S(i) = integral(s, 0, t(i)); % S(t) = ??? sin(1/2?x2) dx
end
% Plot the Cornu spiral
plot(S, C)
xlabel('$S(t)$', 'Interpreter', 'latex', 'FontSize',14);
ylabel('$C(t)$', 'Interpreter', 'latex', 'FontSize',14);
title('Cornu Spiral', 'Interpreter', 'latex', 'FontSize',16);
axis equal;
grid on;
```

Output:



PB-13 Lec 21-27, slide 10 (Interpolation Comparison)

Consider the points (0,0), (1,1) and (2,1)

Use MATLAB to plot the three points and the two interpolating functions.

Matlab Code:

% Given points

```
x_points = [0, 1, 2];
```

```
y_points = [0, 1, 1];
```

% Define x values for smooth curve

```
x_fine = linspace(0, 2, 100);
```

% Quadratic interpolation: $g(x) = -0.5x^2 + 1.5x$

```
y_quad = -0.5*x_fine.^2 + 1.5*x_fine;
```

% Piecewise linear interpolation

```
y_linear = zeros(size(x_fine));
```

```
for i = 1:length(x_fine)
```

```
    if x_fine(i) <= 1
```

```
        y_linear(i) = x_fine(i); % from (0,0) to (1,1)
```

```
    else
```

```
        y_linear(i) = 1; % from (1,1) to (2,1)
```

```
    end
```

```
end
```

% Plotting

```
plot(x_points, y_points, 'ko', 'MarkerFaceColor', 'k'); hold on;
```

```
plot(x_fine, y_quad, 'b-', 'LineWidth', 2); % Quadratic
```

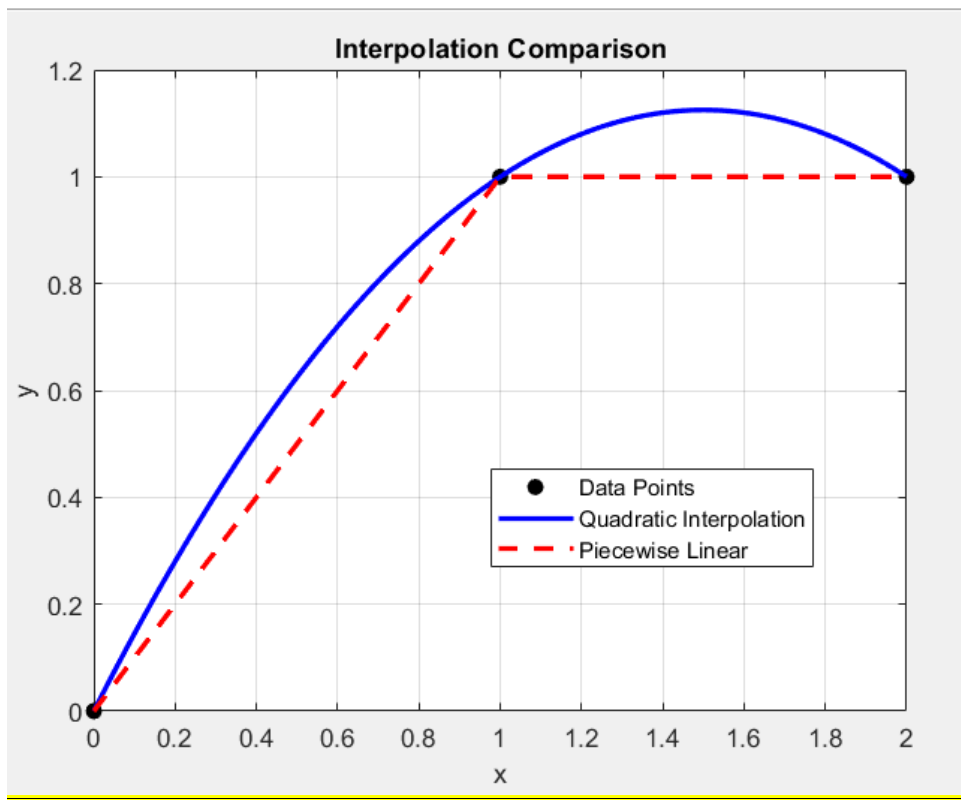
```
plot(x_fine, y_linear, 'r--', 'LineWidth', 2); % Piecewise linear
```

```
legend('Data Points', 'Quadratic Interpolation', 'Piecewise Linear');
```

```
xlabel('x'); ylabel('y'); title('Interpolation Comparison');
```

```
grid on;
```

Output:



PB-14 Lec 28-30, slide 10 (Lorenz Equation)

The Lorenz equations are a system of nonlinear ODEs that pioneered the study of chaos. The Lorenz equations are given by

$$\dot{x} = \sigma(y-x), \quad \dot{y} = x(r-z)-y, \quad \dot{z} = xy-\beta z$$

where σ , β and r are constants. Edward Lorenz studied the solution for $\sigma=10, \beta=8/3, r=28$, and the result is now known as the Lorenz attractor, an example of what is now more generally known as a strange attractor. Compute the Lorenz attractor and plot z versus x and y . Use the Matlab function `plot3.m`. Remove the transient before plotting.

MATLAB CODE:

% Parameters of the Lorenz system

sigma = 10;

beta = 8/3;

r = 28;

% Initial condition and time span

x0 = 1; y0 = 1; z0 = 1;

tspan = [0 100];

% Number of points to discard as transient

ntrans = 2000; % Adjusted based on number of time points

% Set ODE solver options (tight tolerance for accuracy)

options = odeset('RelTol', 1e-6);

% Solve the Lorenz system

[t, xyz] = ode45(@(t, xyz) lorenz_eqs(xyz, sigma, beta, r), tspan, [x0; y0; z0], options);

% Remove transient (here, by removing the first 'ntrans' time steps)

% This assumes t has many points; we can safely cut the first part

x = xyz(ntrans:end, 1);

y = xyz(ntrans:end, 2);

z = xyz(ntrans:end, 3);

% Plot the Lorenz attractor in 3D space

figure;

plot3(x, y, z, 'LineWidth', 1.2);

xlabel('\$x\$', 'Interpreter', 'latex', 'FontSize', 14);

ylabel('\$y\$', 'Interpreter', 'latex', 'FontSize', 14);

zlabel('\$z\$', 'Interpreter', 'latex', 'FontSize', 14);

title('Lorenz Equations', 'Interpreter', 'latex', 'FontSize', 16);

grid on;

% Subfunction for the Lorenz equations

function dxyzdt = lorenz_eqs(xyz, sigma, beta, r)

 x = xyz(1);

 y = xyz(2);

 z = xyz(3);

 dx = sigma * (y - x);

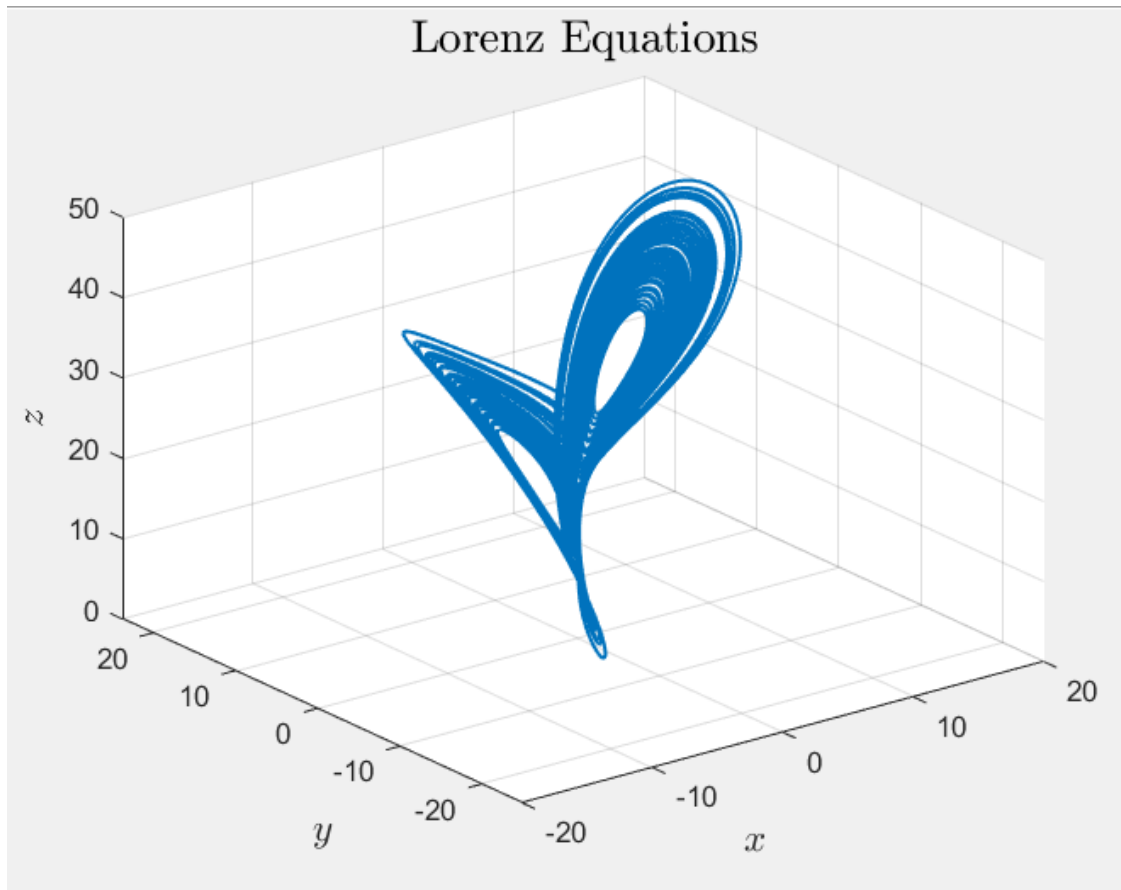
 dy = x * (r - z) - y;

```

dz = x * y - beta * z;
dxyzdt = [dx; dy; dz]; % Return as a column vector
end

```

Output:



PB-15 Lec 30-33, slide 18,27,28 (Laplace Equation)

On a rectangular grid with n_x and n_y grid points, how many interior points are there and how many boundary points? What percentage of grid points are boundary points when $n_x = n_y = 100$, and what percentage when $n_x = n_y = 1000$?

MATLAB CODE:

% Laplace Equation Solver using Finite Differences

```
Lx = 1; Ly = 1;           % Rectangle dimensions
Nx = 100; Ny = 100;       % Number of intervals
nx = Nx + 1; ny = Ny + 1; % Number of grid points
dx = Lx / Nx; dy = Ly / Ny; % Grid spacing
```

```
x = (0:Nx)*dx;           % x-coordinate vector
y = (0:Ny)*dy;           % y-coordinate vector
```

% Build sparse matrix A

```
boundary_index = [1:nx, 1:nx:nx*ny, nx:nx:nx*ny, nx*(ny-1)+1:nx*ny];
diagonals = [4*ones(nx*ny,1), -ones(nx*ny,4)];
A = spdiags(diagonals, [0, -1, 1, -nx, nx], nx*ny, nx*ny);
I = speye(nx*ny);
A(boundary_index,:) = I(boundary_index,:);
```

% Build right-hand side b

```
b = zeros(nx, ny);
b(:,1) = 0;           % Bottom boundary (y=0)
b(1,:) = 0;           % Left boundary (x=0)
b(nx,:) = 0;          % Right boundary (x=1)
b(:,ny) = 4*x.*(1 - x); % Top boundary (y=1)
b = reshape(b, nx*ny, 1); % Convert to column vector
```

% Solve the system

```
Phi = A \ b;
Phi = reshape(Phi, nx, ny); % Reshape solution to 2D grid
```

% Plot the solution

```
[X, Y] = meshgrid(x, y);
v = [0.8, 0.6, 0.4, 0.2, 0.1, 0.05, 0.01]; % Contour levels
contour(X, Y, Phi', v, 'ShowText', 'on'); % Transpose Phi for correct orientation
axis equal;
set(gca, 'YTick', 0:0.2:1);
set(gca, 'XTick', 0:0.2:1);
xlabel('$x$', 'Interpreter', 'latex', 'FontSize', 14);
ylabel('$y$', 'Interpreter', 'latex', 'FontSize', 14);
```

```
title('Solution of the Laplace Equation', 'Interpreter', 'latex', 'FontSize', 16);
```

Output:

